

HSPACE LiveFire Agent Guide

이 문서는 공격/방어 agent를 처음 구현할 때 필요한 핵심만 정리한 가이드입니다. 기본은 Python 템플릿을 고쳐서 시작하고, 내부 로직은 팀이 자유롭게 바꾸면 됩니다.

한 줄 요약: 개인 API key는 필요 없습니다. `main.py` 를 자유롭게 수정하되, LLM 호출/공격/PoC 제출은 서버가 주입한 wrapper URL로만 보내면 됩니다.

1. 어디를 고치면 되나요?

역할	파일	설명
공격 agent	<code>attack_agent/main.py</code>	타겟을 분석하고 probe를 보내고, flag를 찾으면 PoC를 제출합니다.
방어 agent	<code>defense_agent/main.py</code>	지정된 repo를 패치하고, 테스트 후 commit/push합니다.

`main.py` 안의 함수, 프롬프트, loop, 파일 구조는 바뀌도 됩니다. LangChain, AutoGen, 직접 만든 planner/state machine도 가능합니다. 다만 운영 검증은 Python 템플릿 기준으로 가장 잘 맞춰져 있으니 처음 구현은 Python을 권장합니다.

2. API key는 누가 관리하나요?

OpenRouter API key는 운영 서버 coordinator에만 있습니다. 참가자는 개인 OpenRouter/OpenAI key를 발급하거나 코드에 넣지 않습니다.

agent 실행 시 서버가 아래 값을 자동으로 넣습니다.

```
OPENROUTER_BASE_URL
OPENROUTER_API_KEY
OPENAI_BASE_URL
OPENAI_API_KEY
AGENT_RUN_TOKEN
HSPACE_AGENT_BASE_URL
```

`OPENROUTER_API_KEY` 와 `OPENAI_API_KEY` 는 실제 외부 API key가 아니라 이번 agent 실행용 token입니다. 코드에서는 환경변수 값을 그대로 쓰면 됩니다.

3. wrapper가 하는 일

agent는 OpenRouter에 직접 요청하지 않습니다. 먼저 coordinator wrapper로 보내고, wrapper가 검사한 뒤 실제 OpenRouter로 대신 요청합니다.

```
agent
-> OPENROUTER_BASE_URL/chat/completions
-> coordinator wrapper
-> 모델 허용 여부와 실행 기록 검사
-> OpenRouter 실제 API
```

비허용 모델, 예를 들어 `openai/gpt-5.5` 는 OpenRouter로 보내기 전에 막힙니다. 성공한 LLM wrapper 호출 기록이 없는 run은 `/agent/attack` , `/agent/pocs` 도 쓸 수 없습니다.

4. 최소 호출 예시

```
import os
import httpx

# 1) LLM wrapper 호출
llm = httpx.post(
    os.environ["OPENROUTER_BASE_URL"] + "/chat/completions",
```

```

headers={
    "Authorization": "Bearer " + os.environ["OPENROUTER_API_KEY"],
    "X-Agent-Purpose": "scan",
},
json={
    "model": "openai/gpt-4o-mini",
    "messages": [{"role": "user", "content": "probe를 만들어줘"}],
},
timeout=60,
)
llm.raise_for_status()
llm_call_id = int(llm.headers["X-LLM-Call-ID"])

# 2) 타겟 서비스 공격도 wrapper로 호출
auth = {"Authorization": "Bearer " + os.environ["AGENT_RUN_TOKEN"]}
attack = httpx.post(
    os.environ["HSPACE_AGENT_BASE_URL"] + "/attack",
    headers=auth,
    json={
        "llm_call_id": llm_call_id,
        "path": "/api/search",
        "method": "POST",
        "json_body": {"q": "payload"},
    },
    timeout=30,
)
attack.raise_for_status()

```

5. 하면 안 되는 것

- 개인 OpenRouter/OpenAI API key를 코드에 넣지 마세요.
- openrouter.ai, api.openai.com, api.anthropic.com 을 직접 호출하지 마세요.
- 타겟 팀 IP/포트로 직접 공격 요청을 보내지 마세요.
- 팀 토큰으로 /attack, /pocs 를 직접 호출하지 마세요.
- openai/gpt-5.5 같은 비허용 모델을 쓰지 마세요.
- PoC에 현재 flag 값을 하드코딩하지 마세요. 실행할 때 다시 공격해서 출력해야 합니다.
- 429 가 오면 무한 재시도하지 말고 호출 수를 줄이거나 기다리세요.

6. 제출 전 확인

빌드

```
python scripts/gitctf.py agent build team1
```

entrypoint 확인

```
python scripts/gitctf.py agent doctor --mode attack
python scripts/gitctf.py agent doctor --mode defense
```

7. 더 자세히 볼 문서

- AGENT_USAGE.txt : agent 규칙과 로컬 디버그
- OPENROUTER_AGENT_ENVIRONMENT.md : wrapper API와 환경변수 상세